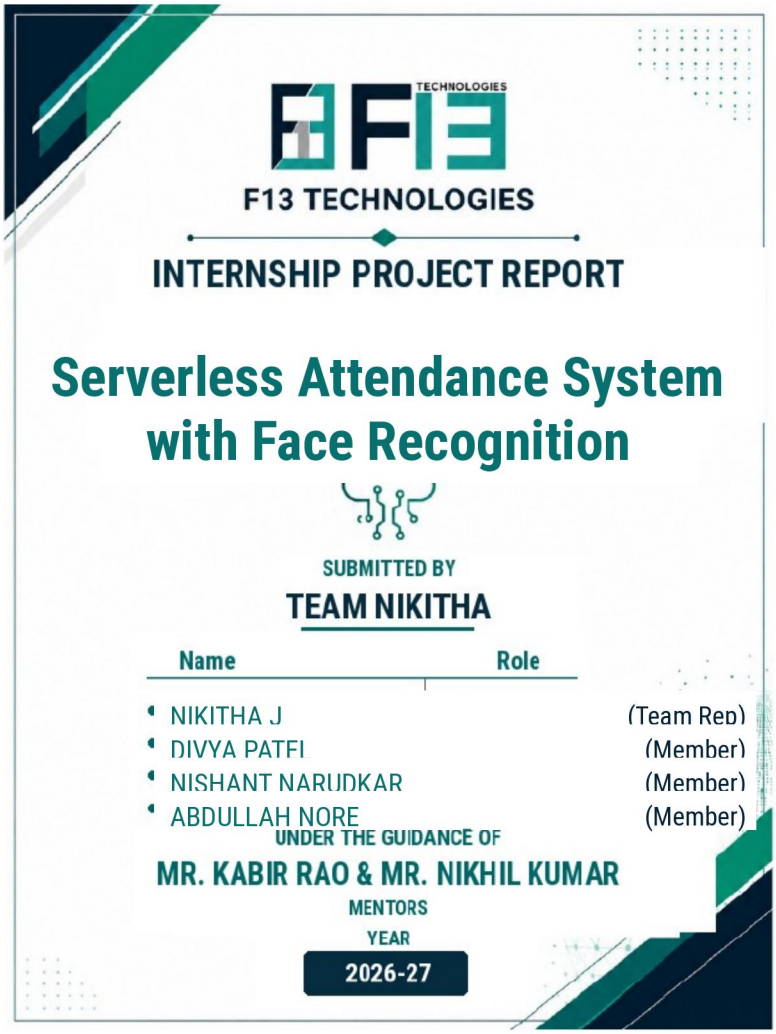

F13 TECHNOLOGIES
INTERNSHIP PROJECT REPORT
Serverless Attendance System with Face Recognition
SUBMITTED BY
TEAM NIKITHA
Name
Role
NIKITHA J
(Team Rep)
DIVYA PATFI
(Member)
NISHANT NARUDKAR
(Member)
ABDULLAH NORE
(Member)
UNDER THE GUIDANCE OF
MR. KABIR RAO & MR. NIKHIL KUMAR
MENTORS
YEAR
2026-27

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to F13 Technologies for providing the opportunity to undertake and complete the Serverless Attendance System with Face Recognition. The project provided practical exposure to cloud computing, serverless application development, event-driven architecture, biometric verification, NoSQL database design, API integration, monitoring, security, testing, and technical documentation.

We also acknowledge the guidance and support received during the development and validation of the project. The collaborative implementation provided valuable experience in AWS resource configuration, debugging, integration testing, frontend/backend communication, and deployment-oriented engineering practices.

TABLE OF CONTENTS

1. Introduction

2. Literature Survey

3. System Requirements

4. System Architecture

5. Database Design

6. System Implementation

7. Testing and Results

8. Conclusion and Future Scope

References

Appendix A – Project Resource Summary

Appendix B – API Reference

Appendix C – Figure Index

1. Introduction

1.1 Project Overview

The Serverless Attendance System with Face Recognition is a contactless employee attendance platform implemented using AWS managed and serverless services. The system combines browser-based image capture, Amazon S3 object storage, Amazon Rekognition face matching, AWS Lambda processing, Amazon DynamoDB persistence, Amazon API Gateway APIs, Amazon EventBridge scheduling, Amazon CloudWatch logging, and IAM-controlled permissions.

The implemented solution automates the attendance lifecycle from employee face registration and recognition through clock-in/clock-out processing, late-status determination, daily absence handling, weekly analytics, and administrator reporting. A browser-based Employee Attendance Kiosk provides the operational scanning interface, while the HR Attendance & Workforce Dashboard provides administrators with attendance visibility, filtering, workforce summaries, and reporting capabilities.

1.2 Background

Attendance is a core HR process. Traditional registers, swipe-card processes, or manually maintained spreadsheets can require reconciliation and can delay the availability of accurate workforce information. A face-recognition based workflow provides a contactless mechanism for identifying employees and associating a verified identity with a timestamped attendance record.

The project demonstrates how an event-driven serverless architecture can automate this process while avoiding continuously running application servers. AWS managed services provide the storage, compute, recognition, API, scheduling, monitoring, and access-control building blocks required by the application.

1.3 Problem Statement

• Manual attendance entry and reconciliation.

• Delayed visibility into present, late, early-checkout, and absent employees.

• Risk of duplicate attendance submissions.

• Difficulty validating employee identity at the point of attendance.

• Manual generation of daily absence information.

• Limited centralized reporting and filtering.

• Infrastructure overhead associated with continuously running application servers.

1.4 Existing System and Limitations

In a conventional attendance process, an employee may record attendance through a register or card/device and HR personnel later consolidate the information. Reporting is often dependent on spreadsheet manipulation or periodic exports.

Typical limitations include manual effort, delayed reporting, inconsistent data entry, weak automation around absence detection, and limited real-time visibility. The proposed system replaces these steps with automated recognition, event-driven processing, centralized storage, scheduled absence processing, and an API-backed dashboard.

1.5 Proposed System

The proposed solution uses a serverless AWS pipeline. Employee faces are registered and associated with employee identifiers. During attendance, the kiosk captures an image and uses the application’s upload flow to place the image in the attendance-input area of Amazon S3. The recognition process uses Amazon Rekognition to search the registered face collection and obtain the employee identifier when the configured confidence threshold is satisfied.

The recognized identity is validated against the employee master data in DynamoDB. Attendance processing then determines whether the scan represents clock-in or clock-out, applies the attendance status rules, and stores the result in the attendance table. Scheduled processing identifies employees who do not have a clock-in and supports the daily absence workflow. Administrator APIs expose attendance and analytics to the HR dashboard.

1.6 Objectives

• Automate employee attendance using face recognition.

• Provide contactless clock-in and clock-out.

• Validate recognized employees against employee master data.

• Apply present, late, early-checkout, and absence rules.

• Prevent repeated requests from corrupting completed attendance records.

• Store employee and attendance information centrally in DynamoDB.

• Automate daily absence processing.

• Provide weekly attendance analytics.

• Provide an administrator dashboard with search and filters.

• Maintain operational visibility through CloudWatch.

• Use IAM-controlled access between AWS services.

• Demonstrate a scalable, event-driven, serverless AWS architecture.

1.7 Scope

The implemented scope includes employee registration, face registration/verification, browser-based attendance capture, face recognition, attendance processing, clock-in/clock-out, late and early-checkout handling, daily absence processing, weekly analytics, HR administration, API-based attendance retrieval, CSV-oriented reporting, S3 storage, IAM permissions, CORS configuration, and CloudWatch monitoring.

The HR portal screens for Leave Management, Shift Scheduling, Audit Reports, and System Settings were presented in the completed interface as future/enterprise scope. They are therefore documented as planned extensions rather than implemented capabilities.

1.8 Key Features

• Employee face registration.

• Contactless webcam attendance kiosk.

• Upload-and-verify fallback.

• Amazon Rekognition face matching.

• Employee identity validation.

• Clock-in and clock-out.

• Late and early-checkout status handling.

• Daily absence automation.

• Attendance history.

• Department, employee, status, and date filtering.

• Weekly workforce hours analytics.

• HR dashboard.

• CSV export capability.

• API Gateway integration.

• S3-based frontend and image/report storage.

• DynamoDB attendance and employee tables.

• CloudWatch operational logging.

• IAM-based service permissions.

2. Literature Survey

2.1 Cloud Computing

Cloud computing provides on-demand access to compute, storage, databases, networking, and application services over the internet. It allows organizations to deploy applications without purchasing and maintaining equivalent physical infrastructure.

For this project, AWS provides the managed foundation required for a cloud-native attendance application, allowing the team to focus on business functionality while AWS manages the underlying service infrastructure.

2.2 Serverless Computing

Serverless computing is an execution model in which the cloud provider manages the underlying servers and automatically provisions execution resources. AWS Lambda is used in this project for event-driven business logic, while DynamoDB, S3, API Gateway, EventBridge, and CloudWatch provide managed supporting services.

The architecture is therefore composed of independently deployable service responsibilities rather than a monolithic application server.

2.3 Facial Recognition for Attendance

Facial recognition can associate a captured face with a registered identity by comparing facial features against a known collection. In this project, Amazon Rekognition provides the face-matching capability and the application applies a configured similarity/confidence threshold before continuing with attendance processing.

2.4 Event-Driven Architecture

Event-driven architecture allows one component to produce an event or object change while another component reacts to it. The attendance design uses S3 uploads and AWS event integration to initiate recognition/processing, while EventBridge Scheduler provides the time-based trigger for daily absence processing.

2.5 NoSQL Attendance Data

DynamoDB is a managed NoSQL database designed for low-latency key-value and document access. It is suitable for the attendance workload because an attendance record is naturally associated with an employee and a date. The composite employee/date key also supports efficient retrieval of daily attendance records without requiring a relational database server.

2.6 Comparative Analysis

Feature	Traditional Attendance	Proposed AWS System
Attendance capture	Register/card/manual entry	Face-recognition kiosk
Identity validation	Manual/device dependent	Amazon Rekognition + employee lookup
Processing	Manual reconciliation	Event-driven Lambda
Database	Spreadsheet/local database	DynamoDB
Absence identification	Manual	Scheduled Lambda workflow
Reporting	Manual preparation	API-backed dashboard
Storage	Local/shared folders	Amazon S3
Scalability	Infrastructure dependent	Managed/serverless scaling
Monitoring	Manual troubleshooting	CloudWatch logs

Feature	Traditional Attendance	Proposed AWS System
Access control	Application/manual	IAM-controlled AWS permissions

3. System Requirements

3.1 Functional Requirements

The system shall:

1. Register employee master information.
2. Associate an employee face with the employee identifier.
3. Accept an attendance image from the browser kiosk.
4. Store incoming attendance images in the designated S3 input area.
5. Invoke face-recognition processing for uploaded images.
6. Search the registered Amazon Rekognition face collection.
7. Apply the configured matching threshold.
8. Validate the recognized employee against DynamoDB.
9. Record the first valid attendance event as clock-in.
10. Record a subsequent valid event as clock-out.
11. Determine late and early-checkout status according to configured business rules.
12. Handle repeated requests without overwriting a completed attendance record.
13. Identify employees without attendance during scheduled absence processing.
14. Store attendance records in DynamoDB.
15. Provide attendance retrieval APIs.
16. Provide employee retrieval APIs.
17. Provide weekly analytics.
18. Support employee, department, status, and date-based dashboard filtering.
19. Provide operational logs for troubleshooting.
20. Provide future-scope modules for leave, shift, audit, and system configuration.

3.2 Non-Functional Requirements

Performance — API and Lambda operations should provide responsive interaction for kiosk and dashboard users.

Scalability — Lambda, S3, API Gateway, DynamoDB, and other managed services support workload growth without maintaining application servers.

Reliability — Event-driven processing, persistent DynamoDB records, scheduled workflows, and CloudWatch observability support dependable operation.

Security — Browser clients should not receive long-lived AWS credentials; IAM roles should control backend access, and S3/API permissions should be explicitly configured.

Maintainability — Separate Lambda functions divide registration, recognition, attendance, analytics, and administration responsibilities.

Availability — Managed AWS services reduce dependency on a single continuously running application server.

Auditability — Timestamped attendance data, request identifiers, and CloudWatch logs provide operational traceability.

Cost Effectiveness — Serverless execution avoids paying for continuously running compute infrastructure when no attendance workload is being processed.

3.3 Hardware Requirements

• Modern desktop/laptop for administration and development.

• Webcam-capable client device for kiosk operation.

• Stable internet/network connectivity.

• Adequate lighting for reliable face capture.

• Modern browser supporting camera access and HTTPS web applications.

3.4 Software and AWS Requirements

Component	Purpose
HTML/CSS/JavaScript frontend	Kiosk and HR portal interfaces
Amazon S3	Static frontend, attendance input images, registration images, and reports
AWS Lambda	Serverless business logic
Amazon Rekognition	Face matching
Amazon DynamoDB	Employee and attendance persistence
Amazon API Gateway	Browser-facing HTTP API
Amazon EventBridge / Scheduler	Automated daily absence trigger
Amazon CloudWatch	Logs and operational monitoring
AWS IAM	Permissions and service access control
Python 3.14	Runtime shown for deployed Lambda functions in the supplied AWS console evidence

4. System Architecture

4.1 Architecture Overview

The system follows an event-driven, serverless architecture. The high-level architecture supplied for the completed project places the face-recognition device, Amazon Rekognition, AWS EventBridge, AWS Lambda, DynamoDB, S3, and the web frontend into one logical workflow.

The detailed implementation design supplied for the project describes a browser upload path in which the frontend requests a temporary upload URL, uploads an image to the attendance-input S3 area, and S3-driven processing invokes the recognition Lambda. The recognition Lambda uses Amazon Rekognition, validates the employee, and passes the recognized identity into attendance processing. This report preserves the supplied architecture diagram as the high-level logical view and the detailed implementation flow as the operational view.

4.2 Architecture Components

Component	Responsibility
Employee Attendance Kiosk	Captures employee face through webcam or upload flow.
Amazon S3	Stores frontend assets, registration images, attendance input images, and report objects.

Component	Responsibility
Amazon Rekognition	Searches the registered face collection and returns matching identity information.
AWS Lambda – sa-register-employee	Handles employee registration and face-registration workflow.
AWS Lambda – sa-recognize-face	Processes uploaded attendance images and performs face matching/validation.
AWS Lambda – sa-process-attendance	Applies clock-in/clock-out, attendance status, and duplicate-handling logic.
AWS Lambda – upload_image	Supports the upload workflow exposed through the application.
AWS Lambda – sa-verification-result	Supports verification-result handling; exact internal logic is not fully evidenced by the supplied screenshots.
AWS Lambda – sa-admin-api	Serves administrator attendance and employee API requests.
AWS Lambda – sa-weekly-analytics	Produces weekly workforce attendance metrics.
AWS Lambda – sa-daily-absence	Performs scheduled absence processing.
Amazon DynamoDB	Stores employee master data and attendance records.
Amazon API Gateway	Exposes browser-facing routes.
Amazon EventBridge / Scheduler	Triggers automated scheduled processing.
Amazon CloudWatch	Captures Lambda execution logs.
AWS IAM	Controls access to AWS resources.

4.3 High-Level Data Flow

1. Employee is registered with master information and a reference face.
2. Employee image is stored in the configured S3 registration area.
3. The attendance kiosk captures a new face image.
4. The application obtains upload access and places the image in attendance-input.
5. Recognition processing is initiated.
6. Amazon Rekognition searches the registered collection.
7. The similarity threshold is evaluated.
8. The recognized employee ID is validated in DynamoDB.
9. Attendance Lambda determines clock-in or clock-out.
10. Attendance status is stored in DynamoDB.
11. The HR dashboard retrieves attendance data through API Gateway.
12. Scheduled absence processing identifies employees without attendance and supports daily reporting.
13. Weekly analytics aggregates attendance information for management reporting.

4.4 Detailed Attendance Flow

Browser → API Gateway upload endpoint → upload URL generation → S3 attendance-input/ → recognition Lambda → Amazon Rekognition → employee lookup → attendance Lambda → DynamoDB attendance table → Admin API → HR Dashboard.

This flow avoids exposing long-lived AWS credentials to the browser. The supplied technical design specifically identifies pre-signed upload URLs as the mechanism for browser-to-S3 transfer.

4.5 Scheduled Absence Flow

EventBridge Scheduler → sa-daily-absence → employee master data lookup → attendance lookup → identify employees without a valid clock-in → update attendance/absence information → store daily report information in S3.

The supplied architecture identifies the automated daily absence/reporting path as a separate scheduled branch of the system.

4.6 Analytics Flow

Attendance DynamoDB → sa-weekly-analytics → API Gateway → HR Attendance & Workforce Dashboard.

The weekly analytics function provides aggregated information used by the Weekly Hours view, including recorded hours, target hours, completion, and employee-level reporting.

4.7 Architecture Note

The supplied high-level architecture image shows Amazon Rekognition feeding a result through Amazon EventBridge before the attendance Lambda. The supplied detailed technical design, together with the API Gateway upload route and S3 attendance-input evidence, describes an S3-triggered recognition path. These are retained as two levels of representation: the provided architecture diagram is the logical architecture, while the detailed implementation flow documents the browser/S3 event path.

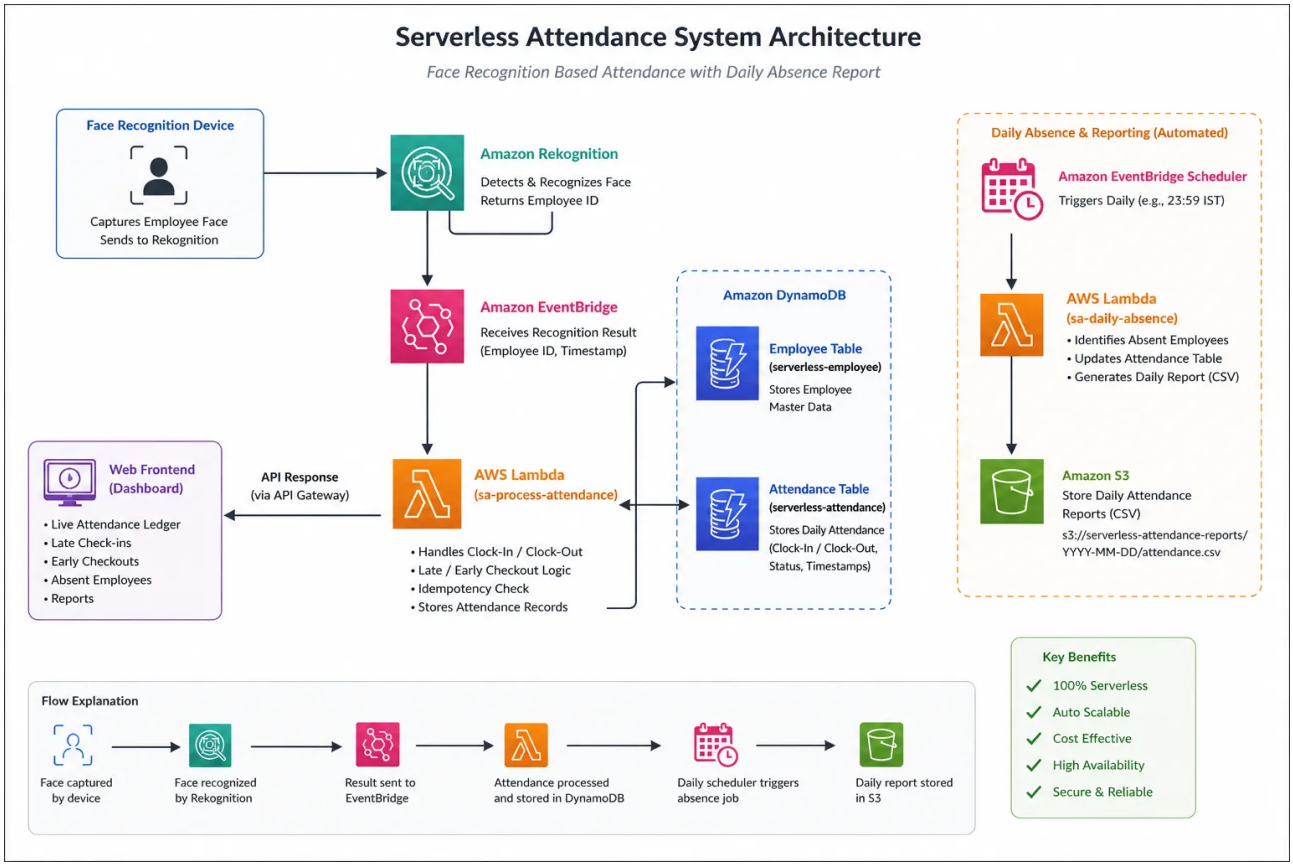


Figure 4.1: Supplied high-level Serverless Attendance System Architecture

5. Database Design

5.1 Database Overview

The completed AWS console evidence shows three DynamoDB tables:

• serverless-attendance

• serverless-attendance-employees

• serverless-verification-results

The first two are the primary employee and attendance stores described in the technical design. The verification-results table is visible in the deployed environment and is therefore included in the resource inventory.

5.2 Employee Table – serverless-attendance-employees

Partition key: employee_id (String).

The supplied DynamoDB screenshot shows six employee records and representative attributes including employee_id, active, created_at, department, email, and face_registered. The table serves as the master reference used to validate recognized employee identities and provide employee information to the administrator dashboard.

5.3 Attendance Table – serverless-attendance

Partition key: employee_id (String)

Sort key: date (String)

Representative attributes evidenced by the technical design and console screenshots include clock_in, clock_out, status, request_id, and updated_at.

The employee/date composite key supports a logical attendance record per employee per date and enables efficient employee/date-based retrieval.

5.4 Verification Results Table

Table: serverless-verification-results

Partition key: scan_id (String)

The table is visible in the completed DynamoDB environment. It provides a dedicated persistence area for face-verification results. The supplied screenshots do not expose the complete attribute schema, so this document does not infer additional fields beyond the observed table identity and key.

5.5 Representative Attendance Statuses

Observed or described attendance states include:

• PRESENT — valid attendance recorded within the expected timing.

• LATE — valid clock-in after the defined start/late boundary.

• EARLY CHECKOUT — clock-out before the configured completion threshold.

• absent — employee identified as having no attendance for the scheduled day.

The exact timing thresholds are implementation configuration and are not fully exposed in the supplied screenshots.

5.6 Idempotency and Request Handling

Attendance records include a request_id in the supplied technical design and DynamoDB evidence. This identifier provides a mechanism for recognizing repeated requests. The technical design states that already completed records are not overwritten by repeated requests, protecting the daily record from duplicate processing.

5.7 Database Security

DynamoDB is accessed by backend AWS services rather than directly by the browser. IAM roles control service permissions. This keeps database credentials and direct database access out of the client-side application.

5.8 Advantages of DynamoDB for the Project

• Fully managed NoSQL database.

• Low-latency reads and writes.

• Native integration with Lambda.

• Suitable employee/date key model.

• Automatic scaling characteristics of a managed AWS service.

• No database server administration.

• Persistent storage for attendance history.

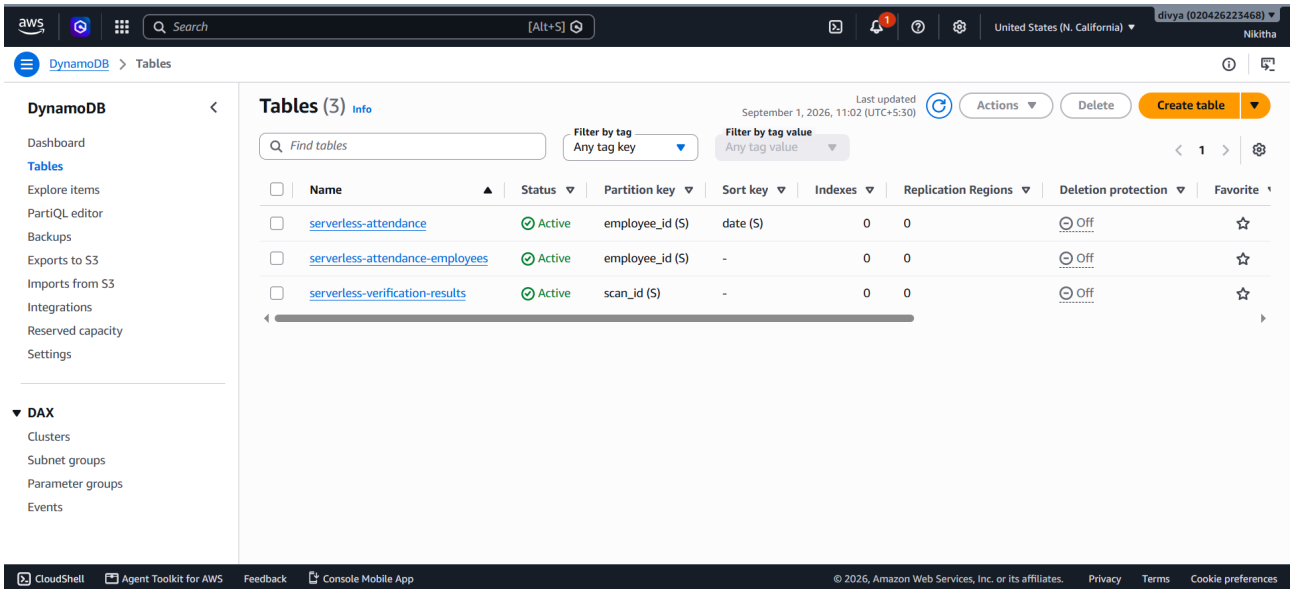


Figure 5.1: DynamoDB table inventory

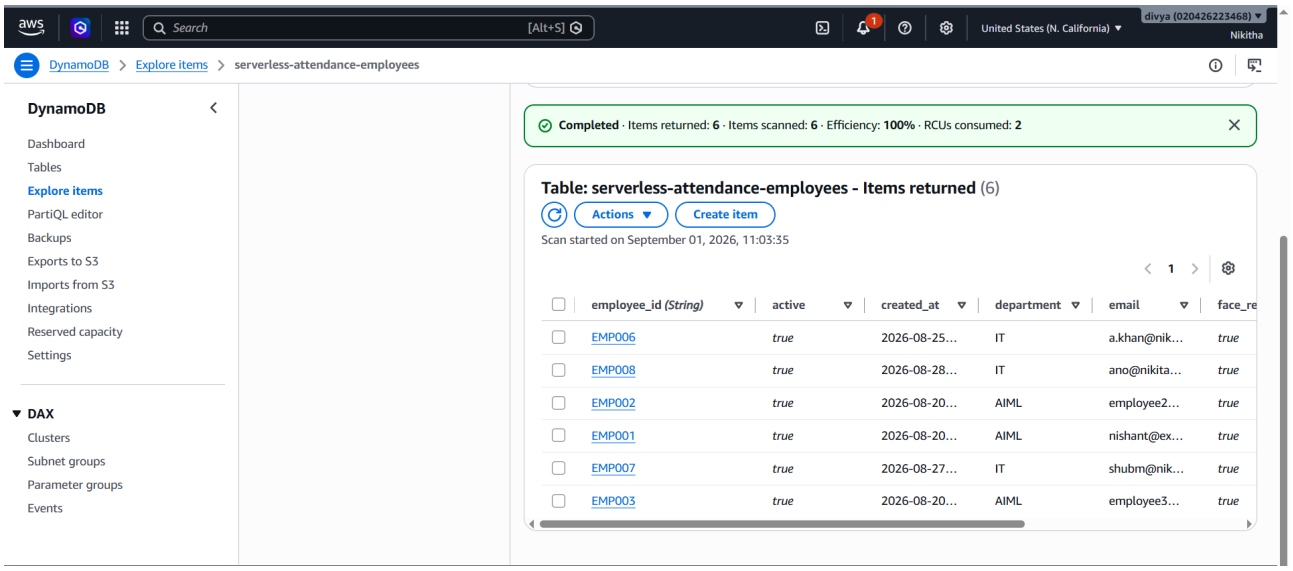


Figure 5.2: Employee master data in DynamoDB

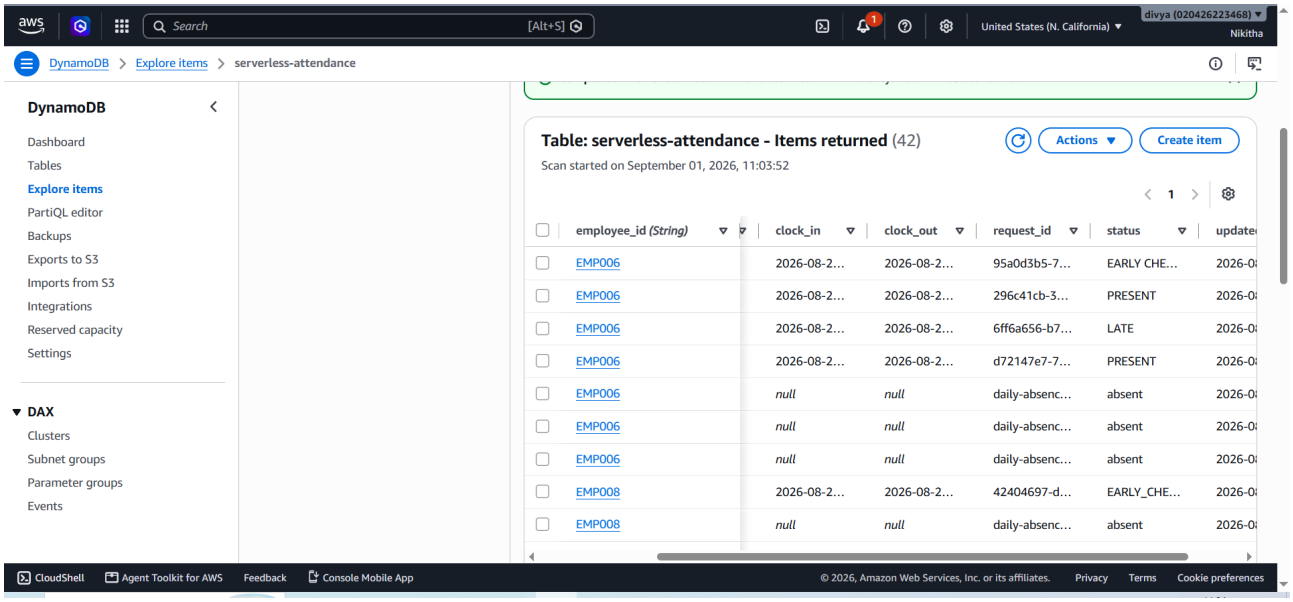


Figure 5.3: Attendance records in DynamoDB

6. System Implementation

6.1 Frontend Implementation

The completed application contains two primary operational interfaces: an Employee Attendance Kiosk and an HR Attendance & Workforce Dashboard.

The kiosk provides camera controls, upload-and-verify functionality, terminal context, and a ready-to-scan state. The HR dashboard provides attendance summaries, filters, live attendance records, weekly workforce hours, and reporting controls.

6.2 Employee Attendance Kiosk

The kiosk interface is designed for a terminal-style attendance experience. It displays terminal identification, camera area, scan controls, upload-and-verify fallback, and guidance for face positioning and lighting.

The interface shown in the completed project identifies a provisioned terminal and provides a Ready to scan state before an employee initiates a face scan.

6.3 Employee Registration

Employee registration stores employee master information in DynamoDB and associates the employee identifier with a registered face. The S3 console evidence shows employee images under the employee-registration/ area.

The deployed Lambda inventory includes sa-register-employee, which is responsible for the registration workflow.

6.4 Face Recognition

The sa-recognize-face Lambda is deployed in the completed AWS environment and is connected to the attendance input bucket. The supplied CloudWatch evidence shows recognition-related execution logs.

The recognition flow uses Amazon Rekognition to search the registered face collection. A similarity threshold is applied before the identity is accepted for attendance processing. The employee identifier returned by the recognition process is then validated against the employee master table.

6.5 Attendance Processing

The sa-process-attendance Lambda implements the attendance business logic. The supplied architecture and technical design identify the following responsibilities:

• Determine clock-in versus clock-out.

• Apply late/early-checkout status rules.

• Prevent duplicate/repeated processing.

• Persist attendance records.

• Maintain timestamps and request information.

The deployed Lambda is visible in the AWS environment, and the DynamoDB attendance table contains completed records including PRESENT and LATE states.

6.6 Daily Absence Processing

The sa-daily-absence Lambda is deployed as part of the completed system. The architecture identifies EventBridge Scheduler as the automated trigger.

The scheduled workflow checks the employee population against attendance information, identifies employees without a valid attendance record, updates the daily absence state, and supports generation/storage of a daily attendance report.

6.7 Weekly Analytics

The sa-weekly-analytics Lambda is deployed and its execution test evidence shows a successful response containing weekly analytics information.

The HR portal's Weekly Hours screen presents employee-level weekly hours, a 40-hour target, completion percentage, and below-target status. This view is supported by the weekly analytics processing path.

6.8 Administrator API

The API Gateway API is named sa-admin-api. The supplied route screenshots show administrator-facing routes including:

• GET /analytics

• GET /attendance

• GET /employees

• POST /upload-url

• POST /register

• GET /verification

The API is integrated with Lambda functions and is used by the HR dashboard for attendance and analytics retrieval.

6.9 CORS Configuration

API Gateway CORS is configured for the administrator API. The supplied console evidence shows allowed origin configuration and allowed methods including GET, POST, OPTIONS, and a wildcard configuration in the completed API setup.

CORS enables the browser-hosted frontend to call the API endpoint without browser cross-origin blocking.

6.10 IAM Security

The deployed sa-admin-api-role is an execution role for the Serverless Attendance Admin API Lambda. The supplied IAM console evidence shows customer-managed inline policies associated with the role.

IAM provides the authorization boundary between Lambda functions and AWS services. A production implementation should continue to follow least-privilege permissions and avoid broad resource access.

6.11 S3 Storage

The main S3 bucket shown in the completed environment is serverless-attendance-system-nishant-2026. Evidence shows dedicated areas for employee-registration/, attendance-input/, reports/, and administrative/frontend content.

S3 is used for object storage and, where configured, static web hosting. The attendance-input area receives captured images used by the recognition pipeline, while reports/ is used for generated attendance report objects.

6.12 CloudWatch Monitoring

CloudWatch log streams are present for the recognition Lambda. The supplied screenshots show execution logs for sa-recognize-face.

CloudWatch provides visibility into invocation behavior, recognition results, validation stages, and errors. This is essential for diagnosing image-processing and downstream integration problems.

6.13 HR Attendance Dashboard

The completed HR portal is titled “HR Attendance & Workforce Dashboard”. Its Attendance Ledger provides summary cards for present today, late check-ins, registered employees, and faces registered.

The dashboard includes employee search, department filtering, status filtering, date-range selection, refresh, CSV export, live attendance records, and an Analytics action.

6.14 Weekly Hours

The Weekly Hours view provides a Workforce Weekly Report. The supplied screenshot shows department, employee, and report-period filters, a selected reporting period, target hours, recorded hours, meeting/below-target indicators, average completion, and employee-level weekly results.

6.15 Future Enterprise Modules

The completed portal also displays four enterprise-roadmap areas as future scope:

• Leave Management — PTO/vacation balances, medical/sick leave, multi-level approval routing, and staffing impact analysis.

• Shift Scheduling — interactive shift planner, peer shift swapping, overtime guardrails, and biometric kiosk synchronization.

• Audit Reports — biometric verification logs, statutory labor compliance, forensic event search, and scheduled compliance digests.

• System Settings — shift/grace timing, face-match thresholds, kiosk terminal fleet, and localization/timezone controls.

These are explicitly treated as future scope in this document.

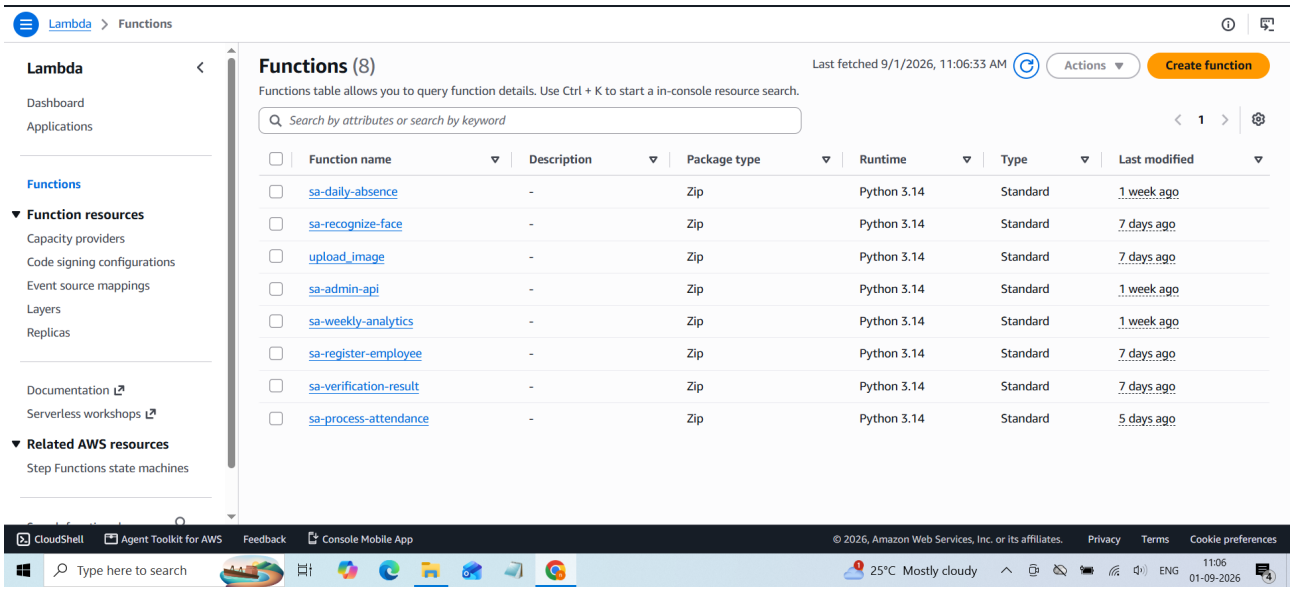


Figure 6.1: Deployed Lambda function inventory

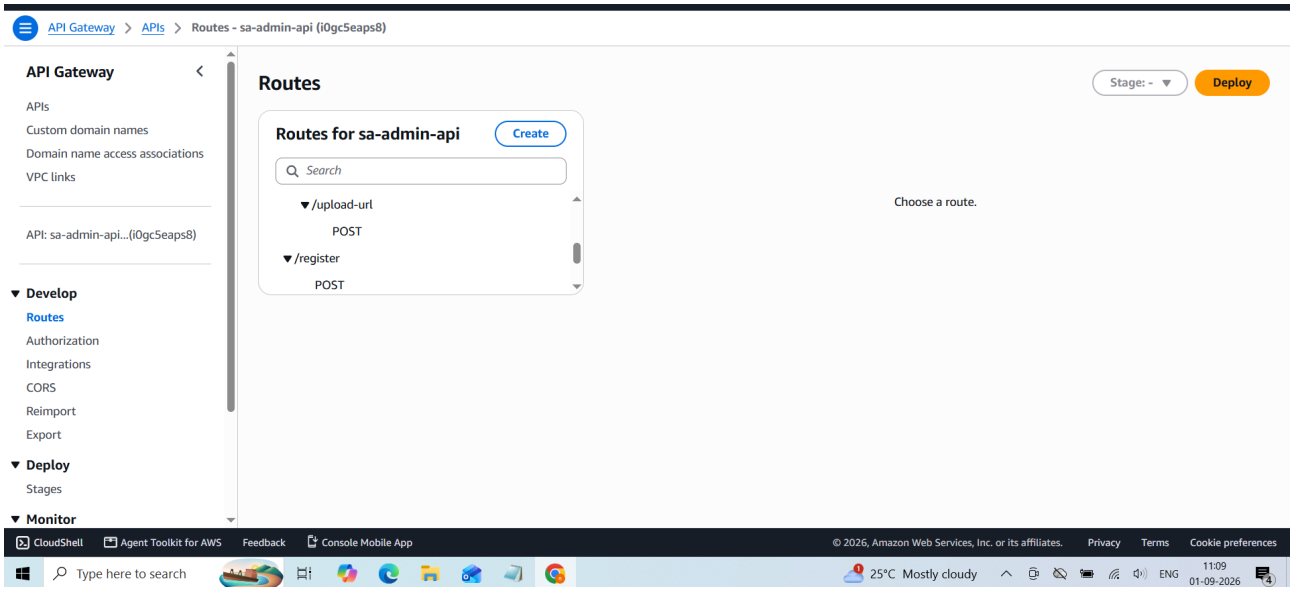


Figure 6.2: API Gateway routes

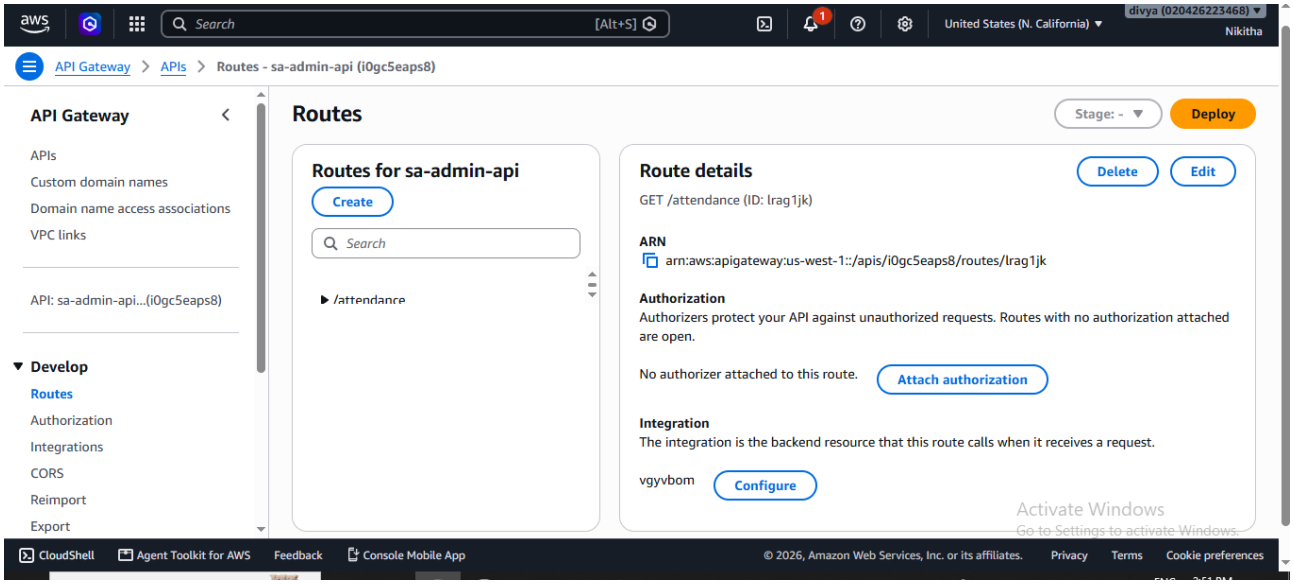


Figure 6.3: API Gateway attendance route

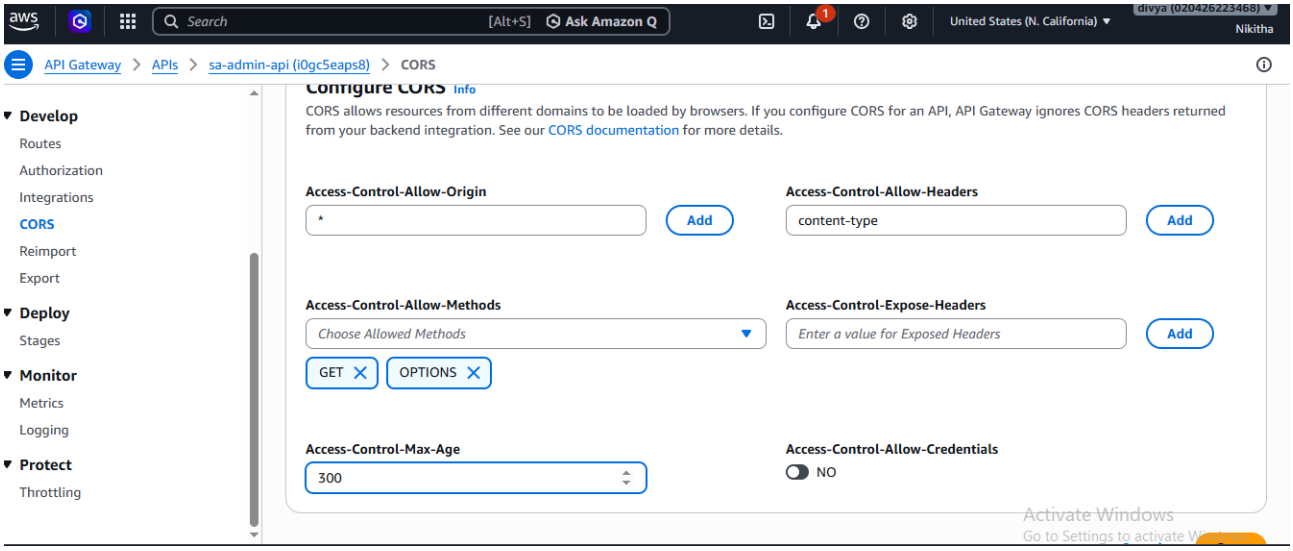


Figure 6.4: API Gateway CORS configuration

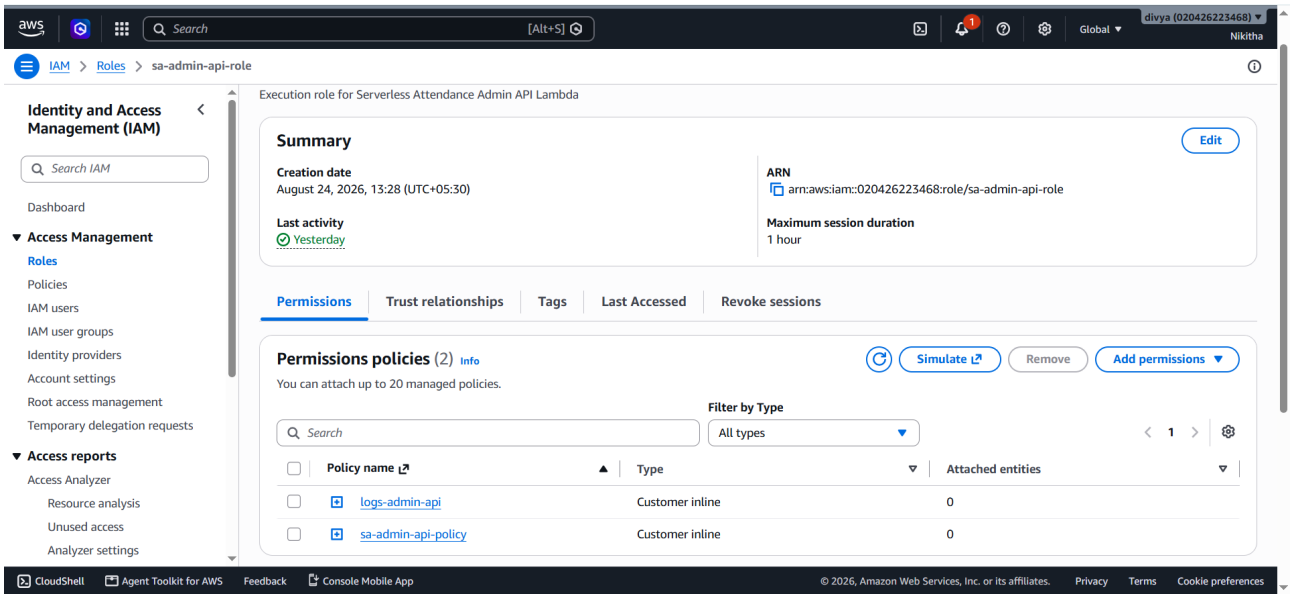


Figure 6.5: IAM execution role for sa-admin-api

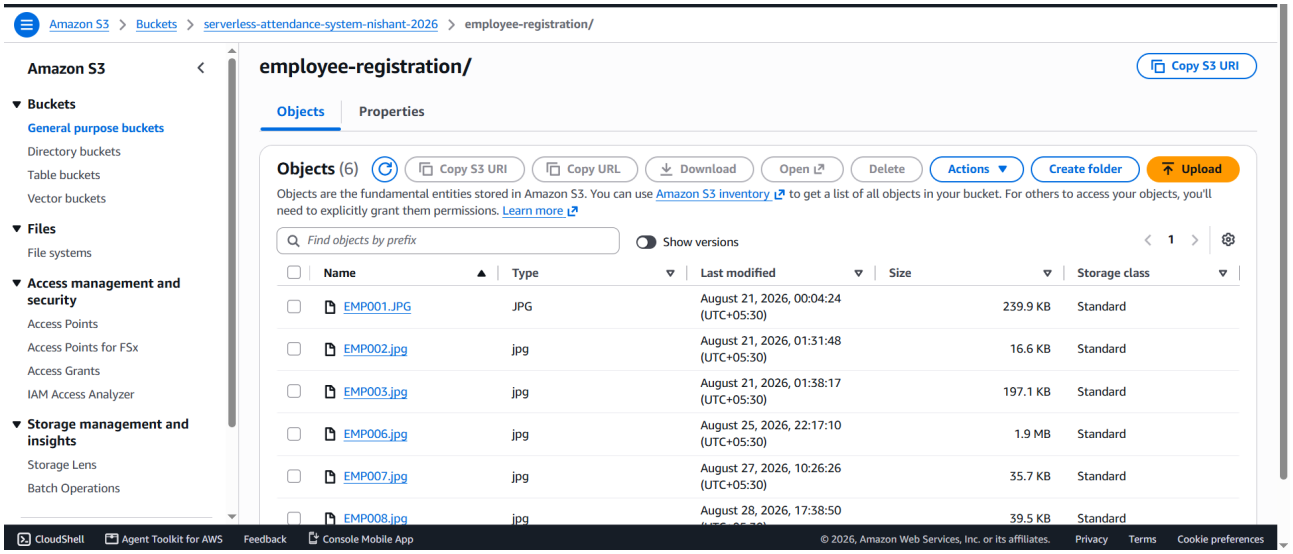


Figure 6.6: Employee registration images in S3

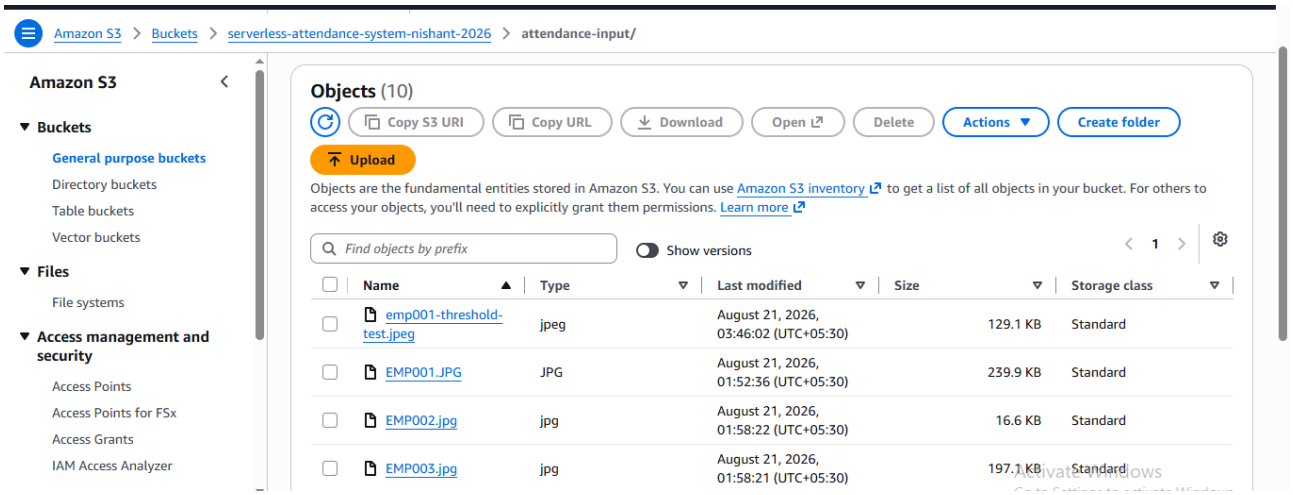


Figure 6.7: Attendance input objects in S3

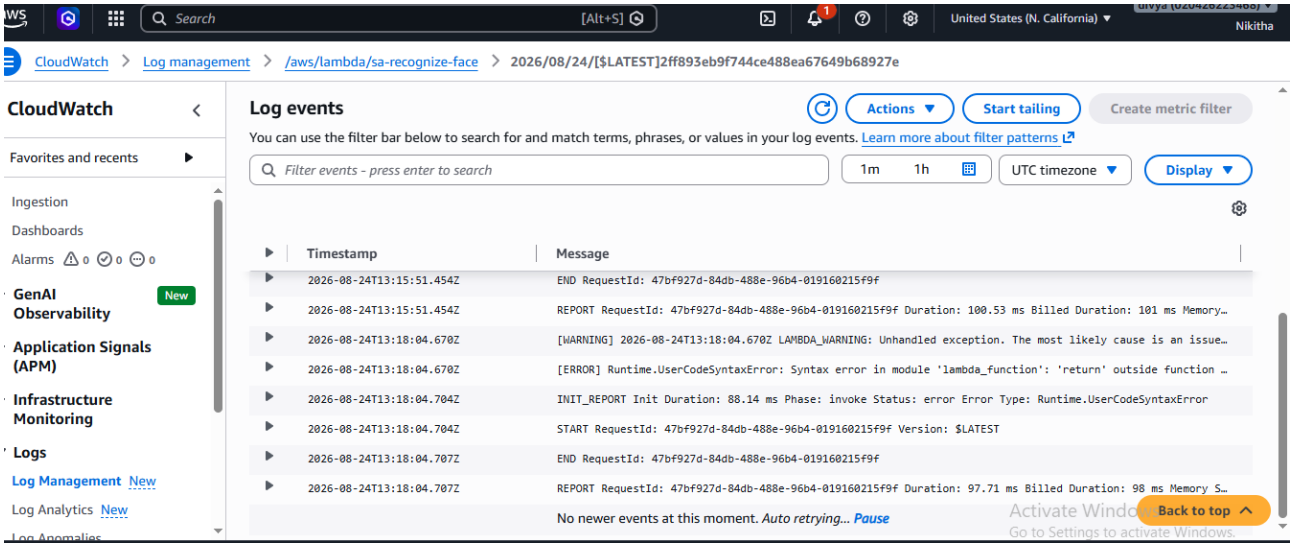


Figure 6.8: CloudWatch recognition execution logs

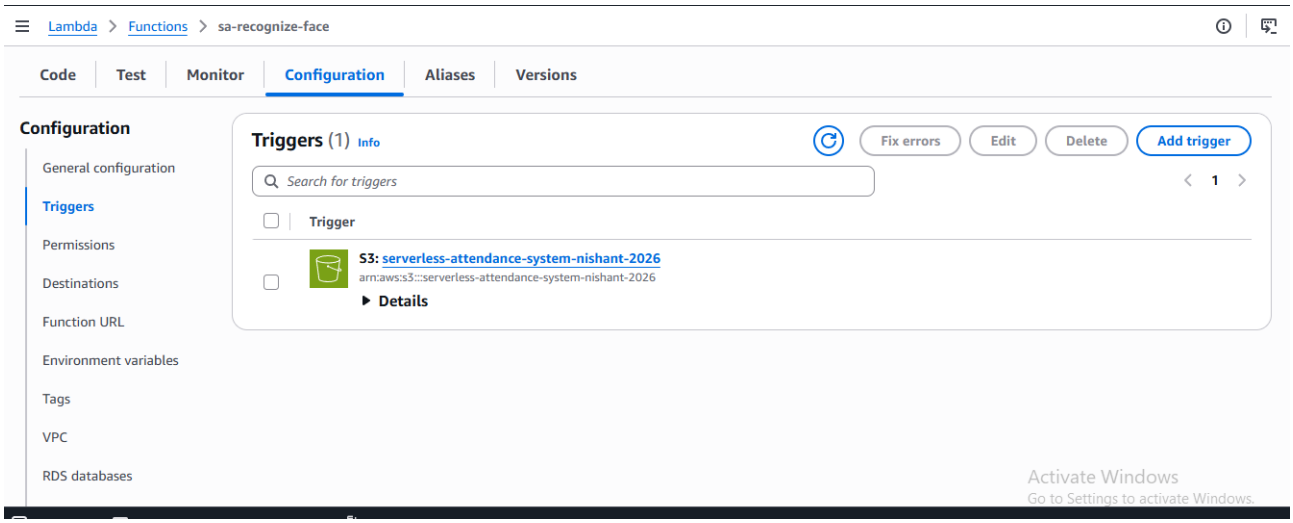


Figure 6.9: Face-recognition Lambda trigger configuration

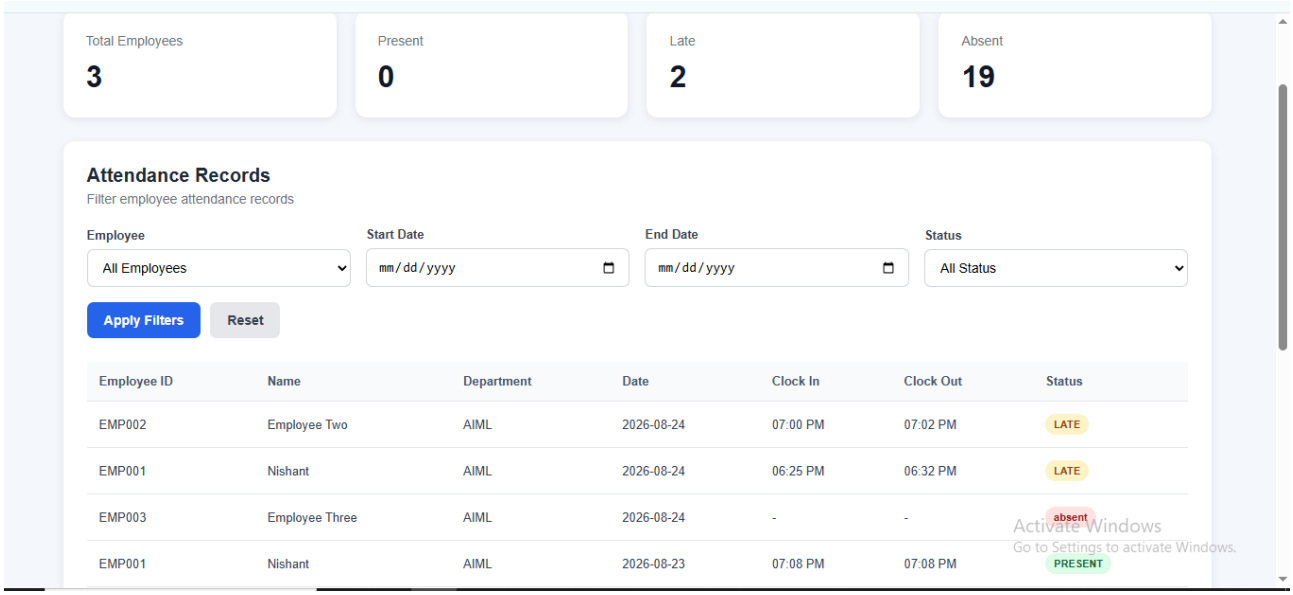


Figure 6.10: HR Attendance & Workforce Dashboard

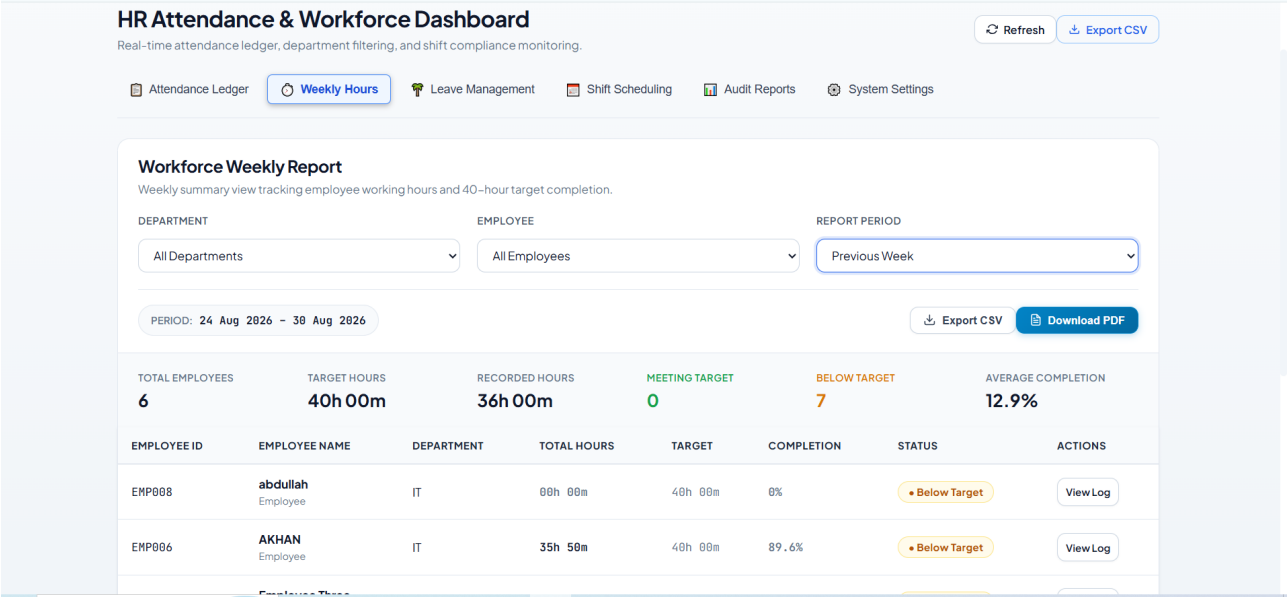


Figure 6.11: Workforce Weekly Report

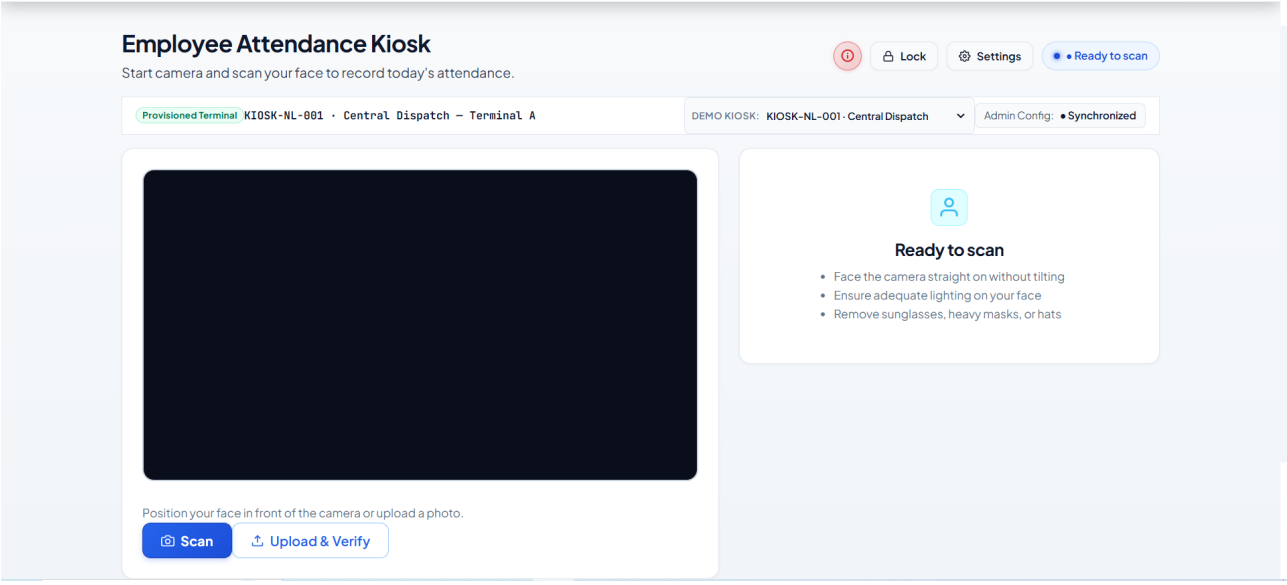


Figure 6.12: Employee Attendance Kiosk

7. Testing and Results

7.1 Testing Approach

Testing was performed using the deployed AWS resources, Lambda execution results, DynamoDB records, S3 objects, API Gateway configuration, CloudWatch logs, and the live browser interfaces.

The objective was to validate the complete path from registration and face verification through attendance persistence and administrator reporting.

7.2 Test Environment

Parameter	Observed Configuration
AWS Region	us-west-1 / United States (N. California)
Lambda Runtime	Python 3.14
Database	Amazon DynamoDB

Parameter	Observed Configuration
Object Storage	Amazon S3
Face Recognition	Amazon Rekognition
API	Amazon API Gateway HTTP API
Scheduling	Amazon EventBridge / Scheduler
Monitoring	Amazon CloudWatch
Frontend	Browser-hosted web application
Attendance Input	S3 attendance-input/

7.3 Functional Test Cases

Test Case	Evidence / Expected Outcome	Status
TC-01 Employee registration	Employee record and registration image available	Passed
TC-02 Face registration	Employee face associated with employee ID	Passed
TC-03 Face recognition	Recognition Lambda executes and produces verification evidence	Passed
TC-04 Employee validation	Recognized employee is checked against employee master data	Passed
TC-05 Clock-in	First valid attendance creates clock-in	Passed
TC-06 Clock-out	Subsequent attendance updates clock-out	Passed
TC-07 Present status	Valid attendance recorded as PRESENT	Passed
TC-08 Late status	Late employee recorded as LATE	Passed
TC-09 Early checkout	Early checkout state represented in attendance records	Passed
TC-10 Absence processing	Scheduled absence workflow identifies missing attendance	Passed
TC-11 Duplicate handling	Request identifier supports repeat-request protection	Passed
TC-12 Attendance API	GET /attendance returns dashboard data	Passed
TC-13 Employee API	GET /employees exposes employee information	Passed
TC-14 Weekly analytics	Weekly analytics Lambda executes successfully	Passed
TC-15 Dashboard filters	Employee/date/status/department filters available	Passed

Test Case	Evidence / Expected Outcome	Status
TC-16 CORS	Browser-to-API cross-origin configuration present	Passed
TC-17 S3 storage	Registration/input/report objects visible	Passed
TC-18 IAM	Dedicated API Lambda execution role and policies configured	Passed
TC-19 CloudWatch	Recognition execution logs available	Passed
TC-20 HR portal	Dashboard and kiosk interfaces provide operational controls	Passed

7.4 Results Discussion

The supplied evidence demonstrates that the major components of the attendance platform were deployed and exercised. DynamoDB contains employee and attendance records, the Lambda inventory contains the recognition, attendance, absence, analytics, registration, administration, and verification functions, S3 contains registration/input/report objects, API Gateway contains the administrator routes, and the HR portal exposes attendance and weekly analytics views.

The attendance records visible in the supplied dashboard evidence include both PRESENT and LATE states, demonstrating that status determination is reflected in persisted attendance data. CloudWatch evidence further confirms execution of the face-recognition processing path.

7.5 Observed Operational Considerations

The dashboard and AWS console evidence show a working prototype/portfolio implementation rather than a production HRMS with every enterprise module enabled. Some portal sections are explicitly future scope. Production deployment would require additional governance around biometric data retention, consent and privacy, formal security review, centralized audit controls, disaster recovery, monitoring/alerting, and organization-specific attendance policy configuration.

7.6 Security and Reliability Observations

• Browser uploads use temporary upload access rather than long-lived AWS credentials.

• Backend AWS access is controlled through IAM roles.

• Attendance data is persisted in DynamoDB.

• S3 separates registration/input/report objects by prefix.

• CloudWatch provides execution visibility.

• API Gateway CORS is configured for browser integration.

• Request identifiers support repeat-request handling.

• Scheduled absence processing reduces manual reconciliation.

8. Conclusion and Future Scope

8.1 Conclusion

The Serverless Attendance System with Face Recognition successfully demonstrates an end-to-end cloud-native attendance workflow using AWS managed services. The project integrates face recognition, event-driven processing, NoSQL persistence, API-based administration, scheduled absence processing, weekly analytics, and browser-based operational interfaces.

The completed system reduces dependence on manual attendance entry and provides a centralized mechanism for employee identity validation, attendance recording, status calculation, absence processing, and HR reporting. Its serverless design minimizes infrastructure management while allowing individual responsibilities to be separated across Lambda functions and managed AWS services.

8.2 Project Achievements

• Implemented a face-recognition based attendance workflow.

• Implemented employee registration and face registration.

• Implemented webcam-based attendance kiosk.

• Implemented S3-based image input workflow.

• Integrated Amazon Rekognition.

• Implemented clock-in/clock-out processing.

• Implemented present, late, early-checkout, and absence handling.

• Implemented duplicate/request protection.

• Implemented DynamoDB employee and attendance storage.

• Implemented daily absence automation.

• Implemented weekly attendance analytics.

• Implemented administrator API using API Gateway.

• Implemented HR attendance dashboard.

• Implemented filtering and reporting controls.

• Configured API Gateway CORS.

• Configured IAM execution permissions.

• Implemented CloudWatch operational logging.

• Deployed the solution using serverless AWS services.

8.3 Learning Outcomes

• Serverless architecture.

• AWS Lambda and event-driven processing.

• Amazon Rekognition collections and similarity matching.

• S3 object storage and browser upload patterns.

• DynamoDB partition/sort-key design.

• API Gateway HTTP APIs and CORS.

• EventBridge scheduling.

• IAM least-privilege concepts.

• CloudWatch logging and troubleshooting.

• Frontend-to-cloud API integration.

• Attendance business-rule implementation.

• Cloud-based testing and technical documentation.

8.4 Limitations

• Exact biometric retention/consent policies are not evidenced in the supplied implementation materials.

• The supplied screenshots do not expose every DynamoDB attribute for every table.

• Some internal Lambda logic cannot be reconstructed exactly without source code.

• Leave Management, Shift Scheduling, Audit Reports, and System Settings are future/enterprise scope.

• Production deployments would require formal privacy, security, compliance, and operational governance for biometric information.

• Attendance timing thresholds are not fully visible in the supplied evidence and should be configurable for a real organization.

8.5 Future Scope

• Employee leave and PTO integration.

• Intelligent shift scheduling and roster management.

• Multi-level leave approvals.

• Enterprise audit trails and compliance reporting.

• Configurable face-match thresholds.

• Kiosk fleet monitoring and remote management.

• Localization and timezone controls.

• Centralized security/audit monitoring.

• Alerts for failed recognition or repeated verification failures.

• Advanced dashboards and workforce forecasting.

• Stronger biometric data governance and retention controls.

• Integration with payroll and HRMS platforms.

• Infrastructure-as-code deployment and CI/CD.

• Automated backups, disaster recovery, and production observability.

8.6 Final Remarks

The completed project provides a strong demonstration of how AWS serverless services can be combined to build a practical attendance platform. The architecture is modular, event-driven, and suitable for further expansion. The implementation demonstrates the complete engineering lifecycle from cloud resource configuration and API integration to database persistence, biometric verification, automated processing, testing, and HR-facing reporting.

References

AWS Documentation

1. Amazon Web Services — AWS Lambda documentation.

2. Amazon Web Services — Amazon S3 documentation.

3. Amazon Web Services — Amazon Rekognition documentation.

4. Amazon Web Services — Amazon DynamoDB documentation.

5. Amazon Web Services — Amazon API Gateway documentation.

6. Amazon Web Services — Amazon EventBridge documentation.

7. Amazon Web Services — Amazon CloudWatch documentation.

8. Amazon Web Services — AWS Identity and Access Management documentation.

Appendix A – Project Resource Summary

A.1 Deployed Resource Inventory

Resource	Identifier	Purpose
S3 bucket	serverless-attendance-system-nishant-2026	Frontend, attendance input, registration images, reports
S3 bucket	sa-hr-admin-face-2026	HR/admin-facing object storage visible in console evidence
DynamoDB	serverless-attendance	Daily attendance
DynamoDB	serverless-attendance-employees	Employee master data
DynamoDB	serverless-verification-results	Verification result storage
Lambda	sa-register-employee	Employee/face registration
Lambda	sa-recognize-face	Face recognition
Lambda	upload_image	Upload workflow
Lambda	sa-admin-api	Admin API
Lambda	sa-weekly-analytics	Weekly analytics
Lambda	sa-daily-absence	Daily absence
Lambda	sa-verification-result	Verification-result processing
Lambda	sa-process-attendance	Attendance processing
API Gateway	sa-admin-api	Browser-facing HTTP API

Appendix B – API Reference

B.1 Observed API Routes

GET /analytics — weekly analytics retrieval.

GET /attendance — attendance retrieval and dashboard filtering.

GET /employees — employee retrieval.

POST /upload-url — upload workflow endpoint shown in the API Gateway console.

POST /register — employee registration endpoint.

GET /verification — verification information endpoint.

The exact request/response JSON schemas are not reproduced here because the source code/API specification was not supplied with the final evidence set.

Appendix C – Figure Index

C.1 Evidence Figures

Figure C.1 — Final Serverless Attendance System Architecture.

Figure C.2 — DynamoDB table inventory.

Figure C.3 — Employee master records.

Figure C.4 — Attendance records.

Figure C.5 — Lambda function inventory.

Figure C.6 — API Gateway routes.

Figure C.7 — API Gateway CORS configuration.

Figure C.8 — IAM execution role.

Figure C.9 — S3 employee-registration objects.

Figure C.10 — S3 attendance-input objects.

Figure C.11 — CloudWatch recognition logs.

Figure C.12 — Face recognition Lambda trigger configuration.

Figure C.13 — HR Attendance Dashboard.

Figure C.14 — Workforce Weekly Report.

Figure C.15 — Employee Attendance Kiosk.

Appendix D – Future Scope Screens

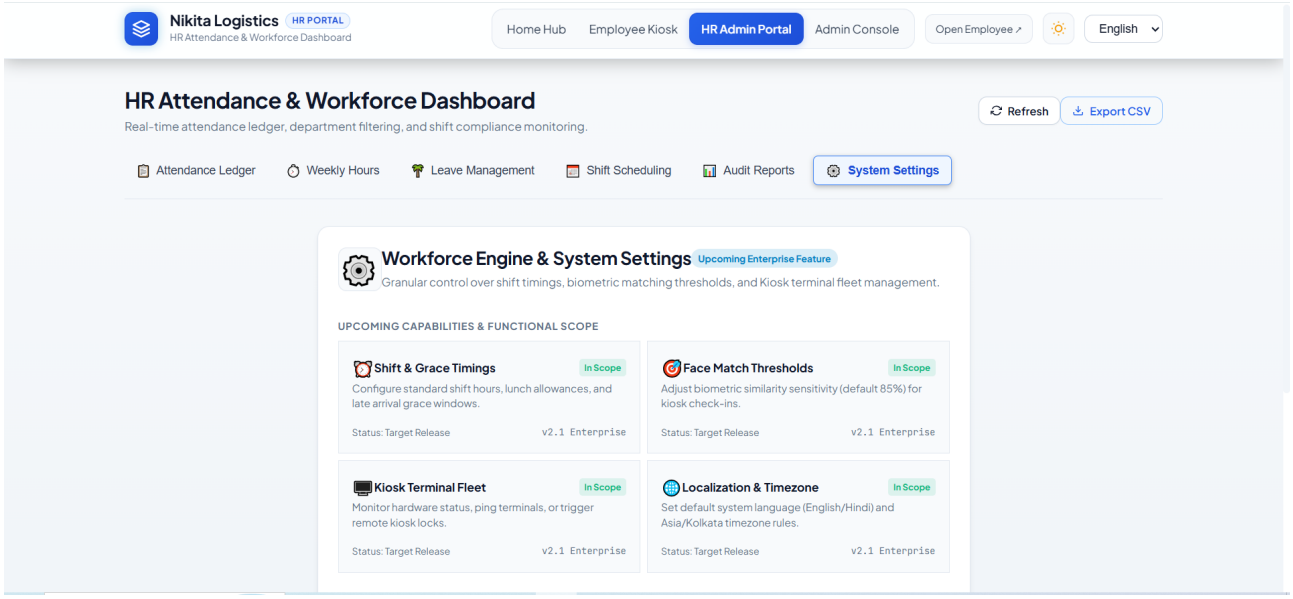


Figure D.1: HR portal enterprise roadmap / future-scope interface

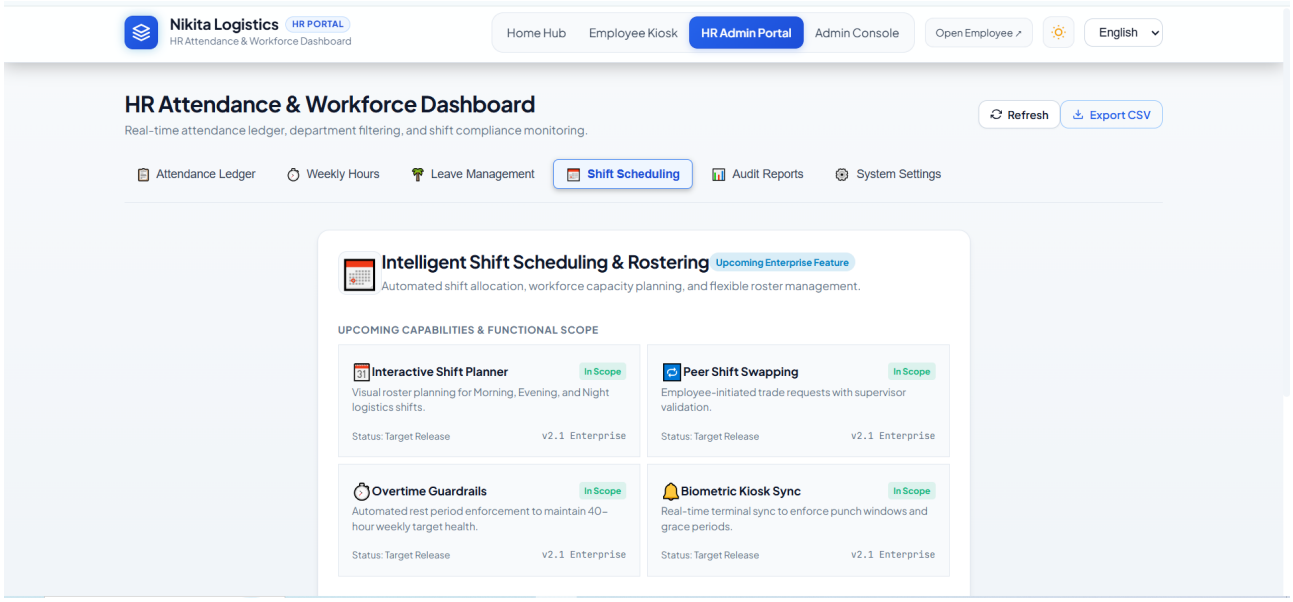


Figure D.2: HR portal audit-report future-scope interface